

Programar fuera del IDE

Cómo crear, guardar y probar tus propias páginas web en tu computadora

Material complementario — Clases 1 a 11 (HTML y CSS)

Hasta ahora escribiste código dentro del entorno de la plataforma: el editor ya tenía el archivo creado, la vista previa se actualizaba sola y el ejercicio te avisaba si el resultado era correcto. Eso sirvió para concentrarte en el lenguaje sin pelear con otras cosas.

Trabajar de verdad es distinto. **Nadie te crea el archivo, nadie te muestra la vista previa y nadie te avisa si te equivocaste.** Esas tres tareas pasan a ser tuyas. Este documento te muestra cómo hacerlas, con dos ejemplos completos que podés abrir, romper y volver a armar.

1. Qué cambia al salir de la plataforma

TAREA	EN EL IDE DE LA PLATAFORMA	EN TU COMPUTADORA
Crear el archivo	Ya estaba creado.	Lo creás vos, con el nombre y la extensión correctos (<code>.html</code> , <code>.css</code> , <code>.js</code>).
Estructura HTML	En varios ejercicios ya venía escrita.	La escribís completa: <code><!DOCTYPE html></code> , <code><html></code> , <code><head></code> , <code><body></code> .
Ver el resultado	Vista previa automática.	Guardás el archivo y lo abrís (o recargás) en el navegador.
Unir HTML y CSS	Ya estaban conectados.	Los conectás vos con <code><style></code> o con <code><link></code> .
Saber si está bien	El ejercicio te corregía.	Mirás la página y usás las herramientas del navegador.

La buena noticia

El lenguaje es exactamente el mismo. Todo lo que escribiste en las clases 1 a 11 funciona igual acá. Lo único nuevo es dónde vive el código y cómo se lo mostrás al navegador.

2. Lo que necesitás

1. **Un editor de texto para código.** Recomendado: *Visual Studio Code* (gratuito, `code.visualstudio.com`). No sirve Word ni Google Docs: agregan formato invisible que rompe el archivo.
2. **Un navegador.** El que ya tenés: Chrome, Firefox o Edge.

3. **Una carpeta.** Creá una carpeta llamada `practica-web` en Documentos o en el Escritorio. Ahí adentro va cada proyecto, en su propia subcarpeta.

Reglas para nombrar archivos y carpetas

- Todo en **minúsculas**: `index.html`, no `Index.HTML`.
- **Sin espacios**: usá guiones. `ficha-proyecto.html`, no `ficha proyecto.html`.
- **Sin tildes ni eñes** en los nombres de archivo: `gestion.html`, no `gestión.html`. (Adentro del archivo sí podés escribir con tildes: para eso está `<meta charset="UTF-8">`.)
- La **extensión importa**. Un archivo llamado `pagina.txt` no se abre como página web aunque tenga HTML adentro.
- La página principal de un sitio se llama `index.html`. Es una convención universal.

Windows: mostrá las extensiones

Si al guardar te queda `index.html.txt`, es porque el sistema oculta las extensiones. En el Explorador: pestaña *Vista* → tildar *Extensiones de nombre de archivo*. En Visual Studio Code esto no pasa.

3. Ejemplo 1 — CSS interno (un solo archivo)

El proyecto entero es **un archivo**. El CSS vive dentro del mismo HTML, encerrado en la etiqueta `<style>`, que va dentro de `<head>`.

```
ejemplo-1-css-interno/  
└─ ficha-proyecto.html      ← HTML + CSS juntos
```

La estructura mínima

```
<!DOCTYPE html>  
<html lang="es">  
<head>  
  <meta charset="UTF-8">  
  <meta name="viewport" content="width=device-width, initial-scale=1.0">  
  <title>Ficha del proyecto</title>  
  
  <style>  
    /* Todo el CSS del ejercicio va acá adentro */  
    body { background: #f5f5f7; }  
  </style>  
</head>  
<body>  
  <h1>Sistema de Turnos</h1>  
</body>  
</html>
```

Qué usa este ejemplo (y en qué clase lo viste)

- Estructura semántica con `header`, `section`, `footer` — clase 5.
- Tabla de módulos con `thead` y `tbody` — clase 3.
- Variables CSS en `:root` usadas con `var()` — clase 8.
- Box model: `padding`, `border`, `margin`, `border-radius` — clase 7.
- Flexbox para el encabezado y la fila de datos — clase 9.

Cómo probarlo

1. Abrí `ficha-proyecto.html` con el editor (clic derecho → *Abrir con* → Visual Studio Code).
2. Hací doble clic sobre el mismo archivo: se abre en el navegador.
3. Dejé las dos ventanas a la vista. Cambiá algo en el editor, guardá con **Ctrl+S** (Mac: **Cmd+S**) y recargá el navegador con **F5**.

Probá esto

Cambiá en `:root` el valor de `--color-primario` por `#7b1fa2` y guardá. Vas a ver que cambia en todos los lugares donde se usa esa variable. Eso es exactamente lo que hace un *token* de diseño.

4. Ejemplo 2 — CSS externo (una carpeta de proyecto)

Acá el proyecto **no es un archivo, es una carpeta**. Cada lenguaje tiene su archivo y su lugar. Así se trabaja en cualquier proyecto real: el HTML dice *qué hay*, el CSS dice *cómo se ve* y el JavaScript dice *qué pasa cuando el usuario hace algo*.

```
ejemplo-2-css-externo/  
├── index.html           ← estructura  
├── css/  
│   └── estilos.css     ← diseño  
└── js/  
    └── script.js       ← comportamiento
```

Las dos líneas que conectan todo

En `index.html`, dentro de `<head>`:

```
<link rel="stylesheet" href="css/estilos.css">
```

Y al final del `<body>`, justo antes de cerrarlo:

```
<script src="js/script.js"></script>  
</body>
```

Sin esas dos líneas, el navegador muestra el HTML sin estilos y sin comportamiento: texto negro sobre fondo blanco. No es un error del CSS, es que nunca se enteró de que existía.

Por qué el `<script>` va al final

El navegador lee de arriba hacia abajo. Si el JavaScript se ejecuta antes de que exista el HTML, busca elementos que todavía no están y falla. Poniéndolo al final, cuando corre ya está todo el `<body>` construido.

Rutas relativas: la parte que más confunde

La ruta se lee **desde la carpeta donde está el archivo que la escribe**.

RUTA	SIGNIFICA
<code>estilos.css</code>	en la misma carpeta que el HTML
<code>css/estilos.css</code>	entrar a la carpeta <code>css</code> y buscarlo ahí
<code>../estilos.css</code>	subir una carpeta y buscarlo ahí
<code>img/logo.png</code>	entrar a la carpeta <code>img</code> y buscar la imagen

Cuidado con la barra

Se usa siempre `/`, incluso en Windows. La barra invertida `\` no funciona en HTML.

Qué agrega este ejemplo

- Grid responsive de tarjetas: una columna en celular, dos a partir de 700 px — clases 10 y 11.
- Clases modificadoras para los estados (`--ok` , `--pruebas` , `--pendiente`) — clase 6.
- Un botón con JavaScript que muestra y oculta los detalles.

El JavaScript, línea por línea

```
const boton = document.querySelector("#btn-detalles");
const lista = document.querySelector("#lista-modulos");

boton.addEventListener("click", function () {
  lista.classList.toggle("modulos--compacto");

  if (lista.classList.contains("modulos--compacto")) {
    boton.textContent = "Mostrar detalles";
  } else {
    boton.textContent = "Ocultar detalles";
  }
});
```

- `document.querySelector("#btn-detalles")` busca en la página el elemento que tiene ese `id` y lo guarda para usarlo después.
- `addEventListener("click", ...)` significa: *cuando alguien haga clic acá, ejecutá esto*.
- `classList.toggle("modulos--compacto")` agrega esa clase si no está, y la saca si ya estaba.
- El diseño no lo decide el JavaScript, lo decide el CSS. En `estilos.css` hay una sola regla que reacciona a esa clase:

```
.modulos--compacto .modulo__detalle { display: none; }
```

La idea que conviene llevarse

El JavaScript no dibuja:

cambia una clase

y el CSS se encarga del resto. Ese reparto de tareas es lo que hace que un proyecto se pueda mantener.

5. ¿CSS interno o externo?

CSS INTERNO (<STYLE>)	CSS EXTERNO (<LINK>)
Un solo archivo, fácil de mandar por correo o subir a una entrega.	Un mismo <code>estilos.css</code> le sirve a 5, 20 o 200 páginas.
Ideal para una prueba rápida o una página única.	Cambiás un color en un lugar y cambia el sitio entero.
Si el sitio crece, el archivo se vuelve enorme e ilegible.	El navegador guarda el CSS en caché: el sitio carga más rápido.
Copiar los estilos a otra página obliga a duplicar código.	Es lo que se usa en cualquier proyecto profesional.

Existe además el **estilo en línea**, escrito en el propio elemento:

```
<p style="color: red">Texto</p> <!-- evitalo -->
```

Funciona, pero mezcla estructura y diseño, es imposible de reutilizar y es difícil de sobrecribir. Se usa solo en casos muy puntuales.

6. Cómo probar y corregir sin que nadie te avise

El ciclo de trabajo

Editar → Guardar (Ctrl+S) → Recargar el navegador (F5) → Mirar. Hacelo seguido, cada pocos cambios. Si escribís 80 líneas y recién ahí probás, cuando algo falle no vas a saber cuál de las 80 fue.

Las herramientas del navegador

Apretá **F12** (o clic derecho → *Inspeccionar*). Te interesan dos pestañas:

- **Elements / Elementos:** señalás un elemento y ves qué reglas CSS le están aplicando, cuáles se están pisando y cuánto padding o margin tiene.
- **Console / Consola:** si el JavaScript falla, el error aparece acá, con el archivo y el número de línea. Si el botón no responde, este es el primer lugar a mirar.

Errores frecuentes

SÍNTOMA	CAUSA HABITUAL	SOLUCIÓN
La página se ve sin ningún estilo	La ruta del <code><link></code> no coincide con la carpeta real, o el archivo tiene otro nombre.	Revisá mayúsculas, guiones y la carpeta <code>css/</code> . Mirá la pestaña Console: avisa si no encontró el archivo.

Se ven tildes raras (Ã¡, Ã±)	Falta <code><meta charset="UTF-8"></code> o el archivo se guardó con otra codificación.	Agregá el meta en <code><head></code> y guardá el archivo como UTF-8.
Cambio el CSS y no pasa nada	No guardaste, o el navegador está usando la versión anterior.	Ctrl+S y recargá forzado: Ctrl+Shift+R (Mac: Cmd+Shift+R).
Se descoloca todo desde cierto punto	Una etiqueta sin cerrar o una llave <code>}</code> faltante en el CSS.	Buscá el último punto donde se veía bien. El editor colorea distinto a partir del error.
El botón no hace nada	Falta el <code><script></code> , la ruta está mal o el <code>id</code> no coincide.	Abrí la consola (F12): el error dice qué línea y qué elemento no encontró.
Aparece barra horizontal en el celular	Falta el meta <code>viewport</code> o un elemento es más ancho que la pantalla.	Verificá el meta y usá <code>max-width: 100%</code> en imágenes.

Antes de pedir ayuda

Mirá la consola, verificá que guardaste y revisá que el nombre del archivo sea exactamente el que escribiste en la ruta. Tres de cada cuatro problemas están ahí.

7. Para practicar

Con los dos ejemplos como referencia, armá tu propio proyecto:

1. Creá la carpeta `practica-web/mi-panel`.
2. Adentro creá `index.html`, la carpeta `css` con `estilos.css`, y la carpeta `js` con `script.js`.
3. Construí una ficha de un proyecto inventado: nombre, descripción, tres datos (versión, responsable, fecha) y al menos cuatro módulos con su estado.
4. Definí tu propia paleta en `:root`: mínimo cuatro variables de color y tres de espaciado.
5. Usá grid o flexbox para las tarjetas y una media query para pasar de una a dos columnas.
6. Sumá un botón que agregue o quite una clase, y que el CSS reaccione a esa clase.

Checklist antes de entregar

- ☐ El archivo principal se llama `index.html` y abre con doble clic.
- ☐ No hay espacios, tildes ni mayúsculas en los nombres de archivo.
- ☐ El HTML tiene `<!DOCTYPE html>`, `lang="es"`, `charset` y `viewport`.
- ☐ No queda ninguna regla de estilo dentro del HTML: todo está en el `.css`.
- ☐ Cerré todas las etiquetas y todas las llaves del CSS.
- ☐ La consola del navegador (F12) no muestra ningún error en rojo.
- ☐ Achiqué la ventana del navegador y la página sigue siendo legible, sin barra horizontal.
- ☐ Si copio la carpeta entera a otra computadora, sigue funcionando igual.

El criterio de fondo

Un proyecto está bien armado cuando alguien más puede copiar la carpeta, abrirla y ver exactamente lo mismo que ves vos, sin que le expliques nada.

Anexo — Código completo de los ejemplos

Los mismos archivos están en la carpeta que acompaña a este documento. Este anexo sirve para leerlos en papel o para reescribirlos a mano, que es la mejor forma de aprenderlos.

Ejemplo 1 — ejemplo-1-css-interno/ficha-proyecto.html

```
<!DOCTYPE html>
<!--
=====
EJEMPLO 1 - CSS INTERNO
=====
Todo el proyecto es UN SOLO archivo: el HTML y el CSS viven
juntos. El CSS va dentro de <style>, y <style> va dentro
de <head>.

Cuándo se usa así:
- páginas de una sola pantalla
- pruebas rápidas
- cuando el estilo no se comparte con otras páginas

Cómo probarlo:
1. Guardá el archivo (Ctrl+S / Cmd+S).
2. Hacé doble clic sobre él: se abre en el navegador.
3. Cambiá algo, guardá y apretá F5 en el navegador.
=====
-->
<html lang="es">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Ficha del proyecto – Sistema de Turnos</title>

  <style>
    /* -----
    1) VARIABLES (clase 8)
    Se declaran una vez en :root y se usan con var() en
    todo el documento. Si cambiás un valor acá, cambia
    en toda la página.
    ----- */
    :root {
      --color-primario: #0057b8;
      --color-texto: #1d1d1f;
      --color-suave: #5b5b60;
      --color-fondo: #f5f5f7;
      --color-superficie: #ffffff;
      --color-borde: #e0e0e6;
      --color-ok: #1a7f4b;

      --espacio-sm: 8px;
      --espacio-md: 16px;
      --espacio-lg: 32px;
      --radio: 10px;
    }

    /* -----
    2) BASE (clases 7 y 8)
    Tipografía, colores generales y el "respiro" de la página.
    ----- */
    body {
      margin: 0;
      font-family: "Segoe UI", Arial, sans-serif;
      line-height: 1.6;
      color: var(--color-texto);
```

```

    background: var(--color-fondo);
    padding: var(--espacio-lg) var(--espacio-md);
}

h1, h2, h3 { line-height: 1.2; margin: 0 0 var(--espacio-sm); }
h1 { font-size: clamp(24px, 5vw, 34px); }
h2 { font-size: 20px; }
h3 { font-size: 16px; }
p { margin: 0 0 var(--espacio-md); }

/* -----
3) CONTENEDOR
Limita el ancho para que el texto no quede kilométrico
en pantallas grandes y centra el contenido.
----- */

.contenedor {
    max-width: 760px;
    margin: 0 auto;
}

/* -----
4) BOX MODEL (clase 7)
La tarjeta usa fondo, borde, padding (espacio interno)
y margin (espacio externo).
----- */

.tarjeta {
    background: var(--color-superficie);
    border: 1px solid var(--color-borde);
    border-radius: var(--radio);
    padding: var(--espacio-md);
    margin-bottom: var(--espacio-md);
}

/* -----
5) FLEXBOX (clase 9)
Encabezado: título a la izquierda, estado a la derecha.
flex-wrap permite que el estado baje si falta lugar.
----- */

.encabezado {
    display: flex;
    flex-wrap: wrap;
    align-items: center;
    justify-content: space-between;
    gap: var(--espacio-sm);
    margin-bottom: var(--espacio-md);
}

.estado {
    background: var(--color-ok);
    color: #ffffff;
    font-size: 13px;
    font-weight: 600;
    padding: 4px 12px;
    border-radius: 999px;
}

/* Fila de datos: tres bloques que comparten el ancho.
Cada uno crece lo mismo gracias a flex: 1. */
.datos {
    display: flex;
    flex-wrap: wrap;

```

```

    gap: var(--espacio-md);
    padding: 0;
    margin: 0;
    list-style: none;
}
.datos li { flex: 1 1 160px; }
.datos .etiqueta {
    display: block;
    font-size: 13px;
    color: var(--color-suave);
    text-transform: uppercase;
    letter-spacing: .04em;
}
.datos .valor { font-weight: 600; }

/* -----
   6) TABLA (clase 3)
   ----- */
table { width: 100%; border-collapse: collapse; }
th, td {
    text-align: left;
    padding: var(--espacio-sm);
    border-bottom: 1px solid var(--color-borde);
}
th { font-size: 13px; color: var(--color-suave); text-transform: uppercase; }

footer {
    color: var(--color-suave);
    font-size: 14px;
    text-align: center;
    margin-top: var(--espacio-lg);
}
</style>
</head>

<body>
<!-- HTML semántico (clase 5): header, main, section, footer -->
<div class="contenedor">

    <header class="encabezado">
        <h1>Sistema de Turnos</h1>
        <span class="estado">En producción</span>
    </header>

    <section class="tarjeta">
        <h2>Descripción</h2>
        <p>
            Aplicación web para reservar y administrar turnos de atención.
            Permite cargar profesionales, definir horarios disponibles y
            enviar recordatorios automáticos.
        </p>

        <ul class="datos">
            <li>
                <span class="etiqueta">Versión</span>
                <span class="valor">2.4.0</span>
            </li>
            <li>
                <span class="etiqueta">Responsable</span>
                <span class="valor">Equipo Backend</span>
            </li>
        </ul>
    </section>
</div>

```

```
        <li>
            <span class="etiqueta">Última entrega</span>
            <span class="valor">14/08/2026</span>
        </li>
    </ul>
</section>
```

```
<section class="tarjeta">
    <h2>Módulos</h2>
    <table>
        <thead>
            <tr>
                <th>Módulo</th>
                <th>Responsable</th>
                <th>Estado</th>
            </tr>
        </thead>
        <tbody>
            <tr>
                <td>Reservas</td>
                <td>M. Ferreyra</td>
                <td>Finalizado</td>
            </tr>
            <tr>
                <td>Notificaciones</td>
                <td>L. Sosa</td>
                <td>En pruebas</td>
            </tr>
            <tr>
                <td>Reportes</td>
                <td>J. Ríos</td>
                <td>En desarrollo</td>
            </tr>
        </tbody>
    </table>
</section>
```

```
<footer>
    <p>Ficha generada por el equipo de desarrollo – Ejemplo con CSS interno.</p>
</footer>
```

```
</div>
</body>
</html>
```

Ejemplo 2 — ejemplo-2-css-externo/index.html

```
<!DOCTYPE html>
<!--
=====
EJEMPLO 2 - CSS EXTERNO (+ un poco de JavaScript)
=====
Acá el proyecto NO es un archivo: es una CARPETA.

    ejemplo-2-css-externo/
    index.html      <- la estructura (esto)
    css/estilos.css <- todo el diseño
    js/script.js    <- el comportamiento

El HTML no tiene ni una sola regla de estilo adentro:
las busca en css/estilos.css con la etiqueta <link>, y busca
el comportamiento en js/script.js con la etiqueta <script>.

Cómo probarlo:
    1. Abrí index.html con doble clic.
    2. Si NO ves colores, el problema es la ruta del <link>.
    3. Si el botón no hace nada, el problema es el <script>.
=====
-->
<html lang="es">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Panel de proyectos – Área de Desarrollo</title>

  <!--
    RUTA RELATIVA: "css/estilos.css" se lee desde la carpeta
    donde está este archivo. Si movés el CSS de lugar, hay
    que corregir esta línea.
  -->
  <link rel="stylesheet" href="css/estilos.css">
</head>

<body>

  <header class="barra">
    <div class="contenedor barra__interior">
      <h1 class="barra__titulo">Panel de proyectos</h1>
      <p class="barra__bajada">Área de Desarrollo – período 2026</p>
    </div>
  </header>

  <main class="contenedor">

    <section class="acciones">
      <!-- El id le da un "nombre" al botón para que el JS lo encuentre -->
      <button id="btn-detalles" class="boton" type="button">Ocultar detalles</button>
      <p class="acciones__ayuda">Probá el botón: lo controla el archivo js/script.js</p>
    </section>

    <!-- GRID (clase 10): las tarjetas se acomodan solas -->
    <section id="lista-modulos" class="modulos">

      <article class="modulo">
```

```

    <h2 class="modulo__titulo">Reservas</h2>
    <p class="modulo__estado modulo__estado--ok">Finalizado</p>
    <p class="modulo__detalle">
        Alta, edición y cancelación de turnos. Incluye control de
        superposición de horarios.
    </p>
</article>

<article class="modulo">
    <h2 class="modulo__titulo">Notificaciones</h2>
    <p class="modulo__estado modulo__estado--pruebas">En pruebas</p>
    <p class="modulo__detalle">
        Envío de recordatorios por correo 24 horas antes del turno.
    </p>
</article>

<article class="modulo">
    <h2 class="modulo__titulo">Reportes</h2>
    <p class="modulo__estado modulo__estado--pendiente">En desarrollo</p>
    <p class="modulo__detalle">
        Estadísticas de asistencia y ausentismo por profesional.
    </p>
</article>

<article class="modulo">
    <h2 class="modulo__titulo">Usuarios</h2>
    <p class="modulo__estado modulo__estado--ok">Finalizado</p>
    <p class="modulo__detalle">
        Registro, inicio de sesión y permisos por rol.
    </p>
</article>

</section>
</main>

<footer class="pie">
    <div class="contenedor">
        <!-- El año lo escribe el JavaScript cuando carga la página -->
        <p>Área de Desarrollo – <span id="anio">2026</span></p>
    </div>
</footer>

<!--
    El <script> va al final del <body>, justo antes de cerrarlo.
    Motivo: cuando se ejecuta, el HTML de arriba ya existe y
    el JavaScript puede encontrar los elementos.
-->
<script src="js/script.js"></script>
</body>
</html>

```

Ejemplo 2 — ejemplo-2-css-externo/css/estilos.css

```
/* =====
ESTILOS DEL PANEL DE PROYECTOS
Este archivo solo tiene CSS: no lleva etiquetas HTML
ni <style>. Se conecta desde index.html con <link>.
===== */

/* 1) VARIABLES (clase 8) ----- */
:root {
  --color-primario: #0057b8;
  --color-texto: #1d1d1f;
  --color-suave: #5b5b60;
  --color-fondo: #f5f5f7;
  --color-superficie: #ffffff;
  --color-borde: #e0e0e6;

  --color-ok: #1a7f4b;
  --color-pruebas: #b06a00;
  --color-pendiente: #8a8a90;

  --espacio-sm: 8px;
  --espacio-md: 16px;
  --espacio-lg: 32px;
  --radio: 12px;
}

/* 2) BASE ----- */
* { box-sizing: border-box; }

body {
  margin: 0;
  font-family: "Segoe UI", Arial, sans-serif;
  line-height: 1.6;
  color: var(--color-texto);
  background: var(--color-fondo);
}

img { max-width: 100%; height: auto; }

.contenedor {
  width: 100%;
  max-width: 960px;
  margin: 0 auto;
  padding: 0 var(--espacio-md);
}

/* 3) BARRA SUPERIOR (flexbox, clase 9) ----- */
.barra {
  background: var(--color-primario);
  color: #ffffff;
  padding: var(--espacio-lg) 0;
}
.barra__interior { display: flex; flex-direction: column; gap: 4px; }
.barra__titulo { margin: 0; font-size: clamp(24px, 5vw, 34px); line-height: 1.1; }
.barra__bajada { margin: 0; opacity: .85; }

/* 4) ACCIONES ----- */
.acciones {
```



```

display: flex;
flex-wrap: wrap;
align-items: center;
gap: var(--espacio-md);
margin: var(--espacio-lg) 0 var(--espacio-md);
}
.boton {
font: inherit;
font-weight: 600;
color: #ffffff;
background: var(--color-primario);
border: none;
border-radius: var(--radio);
padding: 10px 18px;
cursor: pointer;
}
.boton:hover { background: #00448f; }
.acciones__ayuda { margin: 0; font-size: 14px; color: var(--color-suave); }

/* 5) GRID DE MÓDULOS (clases 10 y 11) -----
   Mobile First: una sola columna. Más abajo, con la media
   query, pasa a dos columnas cuando hay ancho suficiente.
   ----- */
.modulos {
display: grid;
grid-template-columns: 1fr;
gap: var(--espacio-md);
margin-bottom: var(--espacio-lg);
}

.modulo {
background: var(--color-superficie);
border: 1px solid var(--color-borde);
border-radius: var(--radio);
padding: var(--espacio-md);
}

.modulo__titulo { margin: 0 0 4px; font-size: 18px; }
.modulo__detalle { margin: var(--espacio-sm) 0 0; color: var(--color-suave); }

.modulo__estado {
display: inline-block;
margin: 0;
font-size: 13px;
font-weight: 600;
color: #ffffff;
padding: 2px 10px;
border-radius: 999px;
}

.modulo__estado--ok { background: var(--color-ok); }
.modulo__estado--pruebas { background: var(--color-pruebas); }
.modulo__estado--pendiente { background: var(--color-pendiente); }

/* 6) CLASE QUE ACTIVA EL JAVASCRIPT -----
   El botón agrega o quita "modulos--compacto" en la lista.
   El CSS decide qué significa eso: ocultar los detalles.
   ----- */
.modulos--compacto .modulo__detalle { display: none; }

/* 7) PIE ----- */
.pie {
border-top: 1px solid var(--color-borde);

```

```
padding: var(--espacio-md) 0;
color: var(--color-suave);
font-size: 14px;
text-align: center;
}

/* 8) RESPONSIVE (clase 11) -----
Solo se cambia lo necesario: la cantidad de columnas.
----- */
@media (min-width: 700px) {
  .modulos { grid-template-columns: 1fr 1fr; }
  .barra__interior { flex-direction: row; align-items: baseline; gap: var(--espacio-md); }
}
```

Ejemplo 2 — ejemplo-2-css-externo/js/script.js

```
/* =====
COMPORTAMIENTO DEL PANEL
Este archivo solo tiene JavaScript: no lleva <script>.
Se conecta desde index.html al final del <body>.
===== */

/* 1) Buscar los elementos con los que vamos a trabajar.
    document.querySelector("#id") devuelve el elemento
    que tiene ese id en el HTML. */
const boton = document.querySelector("#btn-detalles");
const lista = document.querySelector("#lista-modulos");

/* 2) Escuchar el clic.
    "Cuando ocurra el evento click sobre el botón,
    ejecutá estas líneas". */
boton.addEventListener("click", function () {

    /* classList.toggle agrega la clase si no está,
       y la quita si ya estaba. El CSS hace el resto. */
    lista.classList.toggle("modulos--compacto");

    /* El texto del botón tiene que acompañar lo que pasó. */
    if (lista.classList.contains("modulos--compacto")) {
        boton.textContent = "Mostrar detalles";
    } else {
        boton.textContent = "Ocultar detalles";
    }
});

/* 3) Un dato que no conviene escribir a mano:
    el año actual, tomado del reloj de la computadora. */
document.querySelector("#anio").textContent = new Date().getFullYear();
```